CSCE 642: DEEP REINFORCEMENT LEARNING - PROJECT PROPOSAL

GATE-SAC: Uncertainty-Gated Reinforcement Fine-Tuning for Safety-Critical Autonomous Driving

Spoorthy Kumbashi Raghavendra UIN: 437005653

Kritika Bansal UIN: 937006990

Proposed Starting Point

Codebase: The project will be built in Python using the Stable-Baselines3 (SB3) library to handle the core Soft Actor-Critic (SAC) Bellman backups, along with the MetaDrive autonomous driving simulator.

Data & Resources Needed: We will use ScenarioNet to ingest real-world Waymo open driving logs into MetaDrive for the Behavior Cloning phase. This way the policy encounters real human unpredictability giving us genuine edge cases for the RL agent to fix. We will utilize the TAMU HPRC cluster to handle the data ingestion and accelerate the SAC training loops.

What Do You Plan to Do and How Do You Plan to Do It?

The objective is to safely fine-tune an imitation-learned driving policy on safety-critical failure scenarios using a Model-Based Reinforcement Learning (MBRL) pipeline. Industry research relies on closed-loop RL to fix failures in imitation models, but standard model-free RL is sample-inefficient and lacks strict safety bounds.

- **Imitation Base:** We will pretrain a base policy via Behavior Cloning (BC) on expert data so the agent understands fundamental driving mechanics before RL exploration begins.
- **World Model Ensemble:** We will train an ensemble of four MLPs on a real-world replay buffer to predict environment dynamics (next state, reward, and completion flag) directly in vector space.
- **Uncertainty-Gated Imagination & Pessimistic Penalty:** To minimize expensive simulator calls, the policy will train inside the ensemble's imagined environment. We will implement an uncertainty gate that truncates a simulated rollout when prediction disagreement among the four models crosses a threshold, dropping highly uncertain states before they mislead the policy. Because a hard cutoff (like 5 or 7) might blind the agent to states immediately preceding a collision, we will also implement a pessimistic uncertainty penalty. The imagined reward will be mathematically penalized based on the ensemble's disagreement variance using the formula $\tilde{r} = r - \lambda u$, where u is the ensemble prediction variance and λ is a risk-sensitivity coefficient.
- **Policy Optimization:** A standard, randomly initialized Soft Actor-Critic (SAC) agent trains off-policy on a hybrid batch of real and imagined transitions. A Lagrangian cost critic is integrated to enforce a strict safety budget based on collisions per kilometer.

Originality and Attribution:

- The transition ensemble architecture is adapted from Model-Based Policy Optimization (MBPO).
- The pessimistic penalty logic is adapted from MOPO.
- The dynamic uncertainty-gated truncation is adapted from Wu et al. (2021) and MACURA.
- The specific synthesis of imitation fine-tuning, adaptive gating, reward penalization, and Lagrangian constrained MDPs will be coded from scratch using SB3's update loops as a foundation.

In What Sense is Your Problem Sequential?

Autonomous driving is a continuous control Markov Decision Process (MDP). The vehicle's current state and the chosen steering and throttle actions strictly determine its future positional states, spatial relationships with road boundaries, and interactions with dynamic traffic over discrete, sequential time steps.

What is Your Problem's State Space?

A continuous, normalized flat vector. This observation vector encodes the vehicle's kinematics (speed, heading) and surrounding spatial geometry via simulated 1D LiDAR arrays capturing distances to boundaries and other vehicles.

What is Your Problem's Action Space?

The action space defines the exact commands the reinforcement learning agent can send to the car at every time step. The agent outputs a continuous array of two numbers, mathematically denoted as $\mathcal{A} \in [-1, 1]^2$.

These two numbers represent the physical control of the vehicle:

- $a_$**steer (Dimension 1):** Controls the steering wheel angle.
  - -1.0 represents maximum left steering lock.
  - 0.0 represents perfectly straight.
  - 1.0 represents maximum right steering lock.
- $a_$**throttle (Dimension 2):** Combines the gas and brake pedals into a single axis.
  - -1.0 represents maximum braking force.
  - 0.0 represents coasting (neither pedal pressed).
  - 1.0 represents maximum acceleration (flooring the gas pedal).

What is Your Problem's Transition Function?

Real Environment (Ground Truth Transition): The actual physics engine. It calculates the exact next state based on vehicle momentum and Waymo traffic.

- **Function:** It computes the exact rigid-body kinematics of the vehicle, recalculates the 1D LiDAR array based on road boundaries, and updates the positions of the imported Waymo traffic.

- **How it is used:** It is used to collect the "ground truth" data. This real data is used to train the 4-model ensemble, and a portion of it is also fed directly to the SAC agent to ensure the policy stays grounded in true physics. It is also used for the final safety evaluations.

Imagined Environment: The 4-model MLP ensemble (a neural network clone of MetaDrive) predicting (s_{t+1}, r_t, d_t) given (s_t, a_t).

- **Function:** Given a state-action pair (s_t, a_t), the ensemble instantly predicts the numerical values of the subsequent state vector (s_{t+1}), the reward (r_t), and whether the episode has ended (d_t).
- **How it is used:** It generates massive amounts of extra "synthetic" experience. The SAC agent learns by training on a combined batch of the real data and this imagined data, which drastically speeds up the learning process without requiring millions of slow simulator steps.

Why Multiple Models?

Because the imagined environment is just a prediction, it can hallucinate. By using separate models, the system compares their answers. If the models disagree wildly, the system recognizes it as an uncertain situation, triggers the safety gate, and throws away that synthetic data before the SAC agent can learn bad habits from it.

What Reward Function Will You Use?

The setup uses two variations of the reward function depending on where the driving occurs: one for the real MetaDrive simulator, and one modified version for the synthetic imagination phase.

1. Real Environment Reward (Base Driving Objective)

This reward balances route progression with a strict safety constraint, defined as:

$$r(s, a) = r_{\text{progress}} - r_{\text{lane_deviation}} - \lambda c_{\text{collision}}$$

- r_{progress}: Rewards the agent for longitudinal movement toward the destination at a target speed.
- $r_{\text{lane_deviation}}$: Penalizes lateral drift from the lane center to encourage stable steering.
- $c_{\text{collision}}$: A Lagrangian cost multiplier (λ) that dynamically scales up the penalty for crashing $(c_{\text{collision}})$. If the crash rate exceeds the safety threshold, λ increases, forcing the policy to prioritize collision avoidance.

2. Imagined Environment Reward (Pessimistic Penalty)

When the agent trains inside the synthetic 4-model ensemble, neural networks have a tendency to hallucinate "free rewards" in states they haven't learned properly. To prevent the policy from exploiting these errors, we modify the predicted reward:

$$\tilde{r} = r - \lambda u$$

- r **(Predicted Base Reward):** The standard driving reward described above, as estimated by the ensemble.
- u **(Ensemble Uncertainty):** The statistical variance across the models' next-state predictions. If all models predict the exact same outcome, $u \approx 0$. If they diverge, u spikes.

- λ **(Tuning Coefficient):** A scaling factor that controls how risk-averse the agent should be.
- $\tilde{r}$ **(Pessimistic Reward):** The final score fed into the SAC buffer.

Result: This MOPO-style formulation ensures that if the agent imagines an action in an unfamiliar state (like a risky cut-in), the resulting high model variance heavily penalizes the reward. This restricts optimization to safe, well-understood behaviors.

What is the Simplest Possible First Result That You Will Try to Get? What RL Algorithm Will You Use? What Will Be the Baseline?

First Result:

- **Imitation Baseline (Behavior Cloning):** Ingest a curated subset of Waymo Open Dataset scenarios via ScenarioNet and train a 3-layer MLP policy to mimic human steering and throttle commands.
- **Model-Free SAC Floor:** Deploy standard Soft Actor-Critic (SAC) baseline that trains exclusively on real simulator steps without any imagination ensemble.

RL Algorithm: The core learning algorithm is Soft Actor-Critic (SAC), upgraded with a Lagrangian Cost Critic to enforce strict safety limits while learning.

- **Continuous Control:** SAC is the industry standard for driving because it outputs smooth, continuous actions (steering and throttle) rather than jerky, binary commands.
- **Automated Safety Penalties:** Standard RL only focuses on maximizing rewards (like reaching the destination). This setup adds a dedicated cost critic whose job is to track safety violations, such as crashes or off-road excursions. If the vehicle exceeds a predefined safety budget, the system automatically cranks up a penalty multiplier (cost). This mathematically forces the agent to prioritize safe driving.

Baseline: To prove the value of the uncertainty gate, the proposed system will be benchmarked against Fixed-Horizon MBPO (Janner et al., 2019):

- **Fixed-Horizon MBPO Benchmark:** This benchmark uses the exact same 4-model MLP dynamics ensemble but generates synthetic rollouts for fixed step counts regardless of model divergence, blindly feeding hallucinated transitions to the actor-critic. Comparing GATE-SAC against this unpenalized fixed-horizon baseline will directly test whether dynamically truncating high-variance predictions prevents the vehicle from exploiting model errors and adopting dangerous driving behaviors.
- **Testing the Hypothesis:** The project will compare our dynamic approach which actively monitors model disagreement to cut off uncertain predictions and heavily penalize risky states, against the best-performing fixed-horizon baseline. This comparison will prove whether dynamically filtering out bad predictions actually prevents the car from learning dangerous behaviors.

What Will Be the Stretch Goal For Your Project?

- **Vision-Based Perception Pipeline:** Upgrade the state space from 1D kinematic and LiDAR vectors to stacked pixel frames from MetaDrive's onboard cameras, using a Convolutional Neural Network (CNN) or Vision Transformer (ViT) to extract spatial and trajectory features directly from video.
- **Latent World Modeling via RSSM:** Replace the MLP transition ensemble with a Recurrent State Space Model (RSSM). Instead of costly pixel-level predictions, the RSSM compresses visual features into an abstract latent state, allowing the policy to execute rollouts within this learned latent space.

Related Work

- **State-Space Dynamics Ensemble from MBPO (Janner et al., 2019)**
  - *What is reimplemented:* An ensemble of probabilistic/deterministic Multi-Layer Perceptrons (MLPs) trained on the real replay buffer to predict next states (s_{t+1}), rewards (r_t), and termination flags (d_t) directly in the vector observation space.
  - *Difference in our pipeline:* MBPO rolls out models for fixed horizons without uncertainty-based truncation, whereas our pipeline adds an active gate and pessimistic penalty.
- **Adaptive Uncertainty-Gated Truncation (Wu et al., 2021; Pan et al., 2020; MACURA)**
  - *What is reimplemented:* Dynamically truncating imagined rollouts when the ensemble's prediction disagreement (variance across heads) exceeds an uncertainty threshold, dropping unreliable states to prevent error accumulation.
  - *Difference in our pipeline:* We test an adaptive running-percentile threshold rather than a static hand-tuned cutoff, specifically evaluating how the truncation behaves near rare collision events.
- **Pessimistic Reward Penalization from MOPO (Yu et al., 2020)**
  - *What is reimplemented:* Subtracting an uncertainty penalty from the predicted reward at each imagined transition: $\tilde{r} = r - \lambda u$, where u is the ensemble disagreement variance and λ is a tuning coefficient.
  - *Difference in our pipeline:* MOPO applies this in a purely offline setting; we integrate it into an online, model-based off-policy fine-tuning loop to soften the hard cutoff of the uncertainty gate.
- **Lagrangian Constrained SAC (SafeRL-Kit / Ray et al., 2019)**
  - *What is reimplemented:* A two-critic Constrained MDP architecture featuring a primary reward critic alongside an independent cost critic that tracks collisions, with an adaptive Lagrange multiplier maintaining a predetermined safety budget.

References

1. Li, J., Cao, L., Liu, Q., & Xiong, X. (2026). Dreamer-SAC: Off-Policy Learning in Latent World Models for Sample-Efficient Autonomous Driving. *arXiv:2608.10386.*

2. Janner, M., Fu, J., Zhang, M., & Levine, S. (2019). When to Trust Your Model: Model-Based Policy Optimization. *NeurIPS*.

3. Yu, T., Thomas, G., Yu, L., Ermon, S., Zou, J. Y., Levine, S., Finn, C., & Ma, T. (2020). MOPO: Model-based Offline Policy Optimization. *NeurIPS*.

4. Wu, Y., Huang, H., & Lv, C. (2021). Uncertainty-Aware Model-Based Reinforcement Learning with Adaptive Truncation for Autonomous Driving. *arXiv:2106.12194*.

5. Codevilla, F., Müller, M., López, A., Koltun, V., & Dosovitskiy, A. (2018). End-to-end driving via conditional imitation learning. *IEEE International Conference on Robotics and Automation (ICRA)*.

6. Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *International Conference on Machine Learning*.

7. Li, Q., Peng, Z., Feng, L., Zhang, Q., Xue, Z., & Zhou, B. (2022). MetaDrive: Composing Diverse Driving Scenarios for Generalizable Reinforcement Learning. *IEEE TPAMI*.

8. Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research*.